Scientific Density Field Protocols

Stage 11E0a: analysis-owned field surface and zero-copy compatibility adapters

mdstats

2026-07-25

Contents

Purpose and boundary

Stage 11E0a establishes an analysis-owned density field surface before the historical numerical implementation is moved out of `mdstats.plotting`. The owning module is:

`mdstats.analysis.density.protocols`

The stage defines protocols and compatibility adapters only. It does not change cloud-in-cell deposition, periodized Gaussian smoothing, sparse realization, normalization, highest-density-region thresholds, or any numerical field value. Those algorithms remain owned by their existing modules until the deferred density-extraction follow-up D0b. Stage 11E0b is reserved for the registered position-force sample catalog.

The scientific dependency is:

```
current dense or sparse numerical field
        -> ScientificDensityField3D
        -> ScientificDensityFieldAdapter
        -> ScientificDensityFieldBundle
        -> later analysis consumers
```

Rendering dependencies are excluded:

```
mesh arrays
mesh simplification
browser budgets
Plotly traces
HTML serialization
```

Canonical scientific protocol

`ScientificDensityField3D` is a runtime-checkable structural protocol. A field must expose:

- schema, field key, label, and physical units;
- a nonsingular read-only display/reference cell;
- total measure and Gaussian bandwidth;
- smoothing-operator and broadening-metric identifiers;
- storage-backend identity;
- source provenance and immutable metadata;

- logical grid shape and voxel measure;
- realized integral;
- highest-density-region diagnostics and thresholds; and
- a scientific storage summary.

The protocol intentionally contains no rendering method. In particular it has no mesh, trace, browser, figure, or HTML property.

`ScientificPeriodicNodeAccess` is an optional secondary protocol for iterating stored logical nodes and gathering values at periodic logical indices. Dense and block-sparse fields may satisfy it without exposing one common storage layout.

Zero-copy compatibility adapter

`ScientificDensityFieldAdapter` wraps one current numerical field. It validates the scientific protocol and delegates all numerical access. It must not copy or reconstruct the field values.

For an underlying field f, the adapter satisfies:

$$\mathrm{unwrap}(\mathrm{adapt}(f)) \equiv f,$$

where equivalence is object identity, not only numerical equality.

The adapter stores:

- the exact legacy/current field object;
- the temporary numerical-owner module;
- adapter-only immutable metadata;
- an adapter schema; and
- a deterministic contract signature.

The public scientific metadata remain the underlying field metadata. Adapter metadata are separate so the compatibility layer cannot overwrite source, normalization, registration, or kernel provenance.

Contract signature

The adapter signature covers:

- field schema and identifiers;
- units, cell, total measure, bandwidth, and operator identifiers;
- logical-grid and voxel measure;
- realized integral;
- source provenance;
- field metadata;
- storage summary;
- numerical-owner identity; and
- adapter metadata.

It deliberately does not hash every stored voxel. Exact field-content ownership remains with the immutable numerical object until the deferred density-extraction follow-up D0b. Stage 11E0b is reserved for the registered position-force sample catalog. The signature is therefore a **contract/provenance digest**, not a cryptographic digest of the complete scalar array.

Field bundle

`ScientificDensityFieldBundle` is the canonical facade result. It stores:

- one or more adapters with unique field keys;
- source kind;
- scientific-resource signature;
- ownership metadata; and
- a deterministic bundle signature.

A bundle may return the exact current numerical fields through `unwrap_legacy_fields()`. This escape hatch is explicit and transitional. New analysis code should consume the protocol or adapter rather than testing plotting-era concrete classes.

Immutability and validation

The facade fails closed when:

- the field does not satisfy the scientific protocol;
- the display cell is not finite, read-only, 3×3, and nonsingular by the underlying contract;
- scalar measures are non-finite or have invalid sign;
- identifiers are empty;
- logical-grid dimensions are invalid;
- adapter or bundle metadata are not JSON-compatible;
- bundle field keys are duplicated; or
- the resource signature is absent.

Adapter and bundle metadata are recursively frozen. Arrays returned by the underlying scientific field remain read-only.

Public API

```
ScientificDensityField3D
ScientificPeriodicNodeAccess
ScientificDensityFieldAdapter
ScientificDensityFieldBundle
adapt_scientific_density_field
adapt_scientific_density_fields
is_scientific_density_field
has_scientific_periodic_node_access
```

Acceptance requirements

- Dense and sparse current fields satisfy the protocol.
- Adapting a field does not copy its numerical storage.
- Node iteration and gathering are delegated exactly.
- Contract and bundle signatures are deterministic.
- Scientific and adapter metadata remain distinct and immutable.
- No rendering class is part of the analysis protocol.
- Existing numerical field classes remain unchanged.

Method provenance

Stage 11E0a introduces no new scientific estimator. Existing atomic and framework fields continue to use the already documented particle-mesh cloud-in-cell construction adapted from Hockney and Eastwood [1] and the highest-density-region construction described by Hyndman [2]. This stage's protocol, zero-copy adapter, and ownership split are project-specific software architecture.

References

[1] R. W. Hockney and J. W. Eastwood, *Computer Simulation Using Particles*, Taylor & Francis, 1988.

[2] R. J. Hyndman, "Computing and Graphing Highest Density Regions," *The American Statistician* **50**, 120-126 (1996).